Data Collection, Cleaning and Preprocessing for SwiftRoute Logistics Analytics

1. Introduction:

Data quality is an important part of logistics analysis because business decisions depend on accurate and reliable data. In logistics operations, data may contain missing values, duplicate records, incorrect data types, invalid values, and extreme observations. If these issues are not handled properly, they can affect analysis and lead to incorrect business decisions.

This Week 2 project focuses on the data collection, cleaning, and preprocessing stage of the SwiftRoute Logistics project. The same UCI Online Retail Dataset used in Week 1 is used as a public demonstration dataset. The dataset contains transaction-level information such as invoice number, product description, quantity, invoice date, unit price, customer ID, and country.

The main objective is to prepare the dataset for reliable analysis by identifying and resolving data quality issues, handling missing values, detecting outliers, and applying appropriate normalization techniques using Python and pandas. The resulting clean dataset can then be used for further logistics analysis, forecasting, customer segmentation, and business decision-making.

2. Data Collection Simulation:

2.1 Dataset Selection

For this project, UCI Online Retail Dataset is used as a public demonstration dataset. The same dataset was used in Week 1 to maintain consistency across the project.

The dataset contains transaction-level records from an online retail business. Important fields include:

- **InvoiceNo** – identifies a transaction.
- **StockCode** – identifies the product.
- **Description** – product description.

- **Quantity** – number of units purchased.
- **InvoiceDate** – date and time of the transaction.
- **UnitPrice** – price per unit.
- **CustomerID** – identifies the customer.
- **Country** – customer's country.

The dataset can help demonstrate important logistics-related activities such as demand analysis, product movement, customer analysis, and transaction patterns.

2.2 Relevance to SwiftRoute Logistics:

The dataset is useful for demonstrating the data preparation process required before logistics analysis. However, it does not contain actual SwiftRoute operational fields such as vehicle information, delivery time, delivery status, route distance, transportation cost, or warehouse stock levels.

Therefore, the dataset is treated as a public demonstration dataset, while actual SwiftRoute analysis would require operational logistics data collected from orders, warehouses, vehicles, and delivery systems.

2.3 Data Collection Process:

The simulated data collection process consists of:

1. Selecting a relevant public dataset.
2. Loading the dataset into Python using pandas.
3. Inspecting its structure and variables.
4. Identifying missing, duplicate, invalid, and inconsistent records.
5. Preparing the data for cleaning and preprocessing.

This process establishes the foundation for the next stage, where data quality issues are identified and resolved.

3. Data Quality Issues:

Before performing data analysis, the dataset must be checked for errors and inconsistencies. Poor quality data can affect demand analysis, sales calculations, forecasting, and business decisions.

The following data quality issues will be investigated in the UCI Online Retail Dataset.

3.1 Missing Values:

Some records may have missing values in columns such as **CustomerID and Description**. Missing customer IDs can affect customer-level analysis, while missing descriptions can make product identification difficult.

Solution: Identify missing values using pandas. Remove records with missing values only when the missing information is essential for the intended analysis.

3.2 Duplicate Records:

Duplicate rows may cause transactions or quantities to be counted more than once, resulting in inaccurate analysis.

Solution: Use pandas **duplicated()** function to identify duplicate rows and **drop_duplicates()** to remove exact duplicates after checking them.

3.3 Invalid Values:

The dataset may contain zero or negative quantities, non-positive unit prices, and cancelled transactions. Negative quantities may represent returns or adjustments, so they should not automatically be treated as errors.

Solution: Inspect these records and separate valid returns and cancellations from ordinary sales, depending on the purpose of the analysis.

3.4 Inconsistent Data Types:

Columns may be stored using data types that are unsuitable for analysis. For example, **InvoiceDate** must be converted into a datetime format for time-based analysis.

Solution: Check columns data types and use pandas function such as **pd.to_datetime()** to convert them where necessary.

3.5 Outliers:

Outliers are observations that differ substantially from the majority of the data. For example, unusually high quantities or unit prices may influence average sales and statistical analysis.

Solution: Use descriptive statistics, box plots, and the Interquartile Range (IQR) method to identify unusual observations. Investigate them before deciding whether to retain, transform, or exclude them.

Identifying these issues is the first step toward preparing a reliable dataset for further analysis.

4. Data Cleaning and Handling Missing Values:

Data cleaning is the process of identifying and correcting errors, missing values, duplicate records, and invalid data. It helps improve data quality and prepares the dataset for reliable analysis.

4.1 Loading and inspecting the Dataset:

The dataset is loaded using pandas. Initial inspection helps understand its structure, data types, missing values, and duplicate records.

```
import pandas as pd
import numpy as np

df = pd.read_excel("Online Retail.xlsx")

print(df.head())
print(df.shape)
print(df.info())
print(df.isnull().sum())
print("Duplicate rows:", df.duplicated().sum())
```

These commands display sample records, dataset dimensions, column data types, missing-value counts, and duplicate rows.

4.2 Handling Missing Values:

Missing values can affect customer analysis and product identification. First, the number of missing values in each column is checked.

```
print(df.isnull().sum())
```

Records with missing Customer ID can be excluded from customer-level analysis, while records with missing Description can be excluded from product-description analysis. Missing values should not be filled blindly because this may introduce incorrect information.

For a basic cleaned sales dataset records without a product description or customer ID may be retained if those fields are not essential to the analysis. The treatment should depend on the analysis objective.

4.3 Removing Duplicate Records:

Exact duplicate rows can lead to incorrect totals if counted multiple times.

```
df = df.drop_duplicates()

print("Dataset shape after removing duplicates:", df.shape)
```

The drop_duplicates() function removes repeated rows while retaining one copy of each record.

4.4 Correcting Data Types:

The InvoiceDate column must use a datetime data type for time-based analysis.

```
df["InvoiceDate"] = pd.to_datetime(
    df["InvoiceDate"], errors="coerce"
)

print(df.dtypes)
```

The errors= “coerce” option converts unrecognised date values into missing value (NaT), which should then be inspected.

4.5 Handling Invalid Transactions:

The dataset may contain cancelled invoices, negative quantities, and non-positive unit prices. Negative quantities can represent returns, so these records must be inspected before removal.

```
cancelled = df["InvoiceNo"].astype(str).str.startswith("C")
print("Cancelled invoices:", cancelled.sum())
print("Non-positive quantities:", (df["Quantity"] <= 0).sum())
print("Non-positive prices:", (df["UnitPrice"] <= 0).sum())
```

For an analysis focused on ordinary sales, cancelled invoices and records with non-positive quantities or prices can be excluded from the sales dataset.

4.6 Checking the Cleaned Dataset:

Finally, the cleaned dataset is checked for remaining missing values and duplicates.

```
print("Missing values:")
print(sales_df.isnull().sum())

print("Duplicate rows:", sales_df.duplicated().sum())
print(sales_df.head())
print(sales_df.describe())
```

This final inspection helps confirm the dataset's condition before proceeding to outlier detection and normalization.

5. Outlier Detection and Treatment:

Outliers are values that differ considerably from observations in a dataset. In the Online Retail Dataset, unusually high quantities or unit prices may affect statistical summaries and sales analysis. However, an outlier is not necessarily an error, as it may represent a genuine bulk purchase.

5.1 Interquartile Range (IQR) Method:

The IQR method identifies potential outliers using the first quartile (Q1) and third quartile (Q3).

- IQR = Q3 − Q1
- Lower bound = Q1 − 1.5 × IQR
- Upper bound = Q3 + 1.5 × IQR

.

Values outside these bounds are flagged as potential outliers and should be investigated.

5.2 Python Implementation:

```
Q1 = sales_df["Quantity"].quantile(0.25)
Q3 = sales_df["Quantity"].quantile(0.75)

IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = sales_df[
   (sales_df["Quantity"] < lower_bound) |
   (sales_df["Quantity"] > upper_bound)
]

print("Potential quantity outliers:", len(outliers))
print("Lower bound:", lower_bound)
print("Upper bound:", upper_bound)
```

This code identifies unusually low or high quantities using the IQR method. The same approach can be applied to **UnitPrice** after checking for valid positive values.

5.3 Treatment of Outliers:

Potential outliers should be examined before any changes are made. Genuine bulk purchases should generally be retained, while

confirmed data-entry errors may be corrected or excluded when reliable evidence is available.

For this project, the original cleaned sales dataset will be retained, and potential outliers will be flagged for investigation rather than automatically deleted.

5.4 Importance for Logistics Analysis:

Outlier detection helps identify unusual transaction quantities and prices that may influence demand estimates and sales summaries. Reliable treatment of outliers supports better forecasting and business decisions.

6. Data Normalization and Standardization:

Data normalization and standardization are preprocessing techniques used to bring numerical features onto comparable scales. They can improve the performance of certain machine learning algorithms, especially those based on distance calculations, such as K-Means clustering.

6.1 Min-Max Normalization:

Min-Max normalization scales numerical values to a specified range, usually between 0 and 1.

Formula:

Normalized value = (Value - Minimum) / (Maximum - Minimum)

6.2 Standardization:

Standardization transforms numerical values using the mean and standard deviation, producing features with a mean of approximately 0 and a standard deviation of approximately 1.

Formula:

Standardized value = (Value - Mean) / Standard Deviation

6.3 Python Implementation:

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# Select numerical features for demonstration
features = sales_df[["Quantity", "UnitPrice"]].copy()

# Min-Max normalization
minmax_scaler = MinMaxScaler()
normalized_data = minmax_scaler.fit_transform(features)

normalized_df = pd.DataFrame(
    normalized_data,
    columns=["Quantity_Normalized", "UnitPrice_Normalized"],
    index=features.index
)

print(normalized_df.head())

# Standardization
standard_scaler = StandardScaler()
standardized_data = standard_scaler.fit_transform(features)

standardized_df = pd.DataFrame(
    standardized_data,
    columns=["Quantity_Standardized", "UnitPrice_Standardized"],
    index=features.index
)

print(standardized_df.head())
```

6.4 Importance of Scaling:

Normalization and standardization prevent numerical features with larger scales from dominating distance-based algorithms. They are useful when preparing data for customer segmentation and other machine learning tasks.

Scaling is not always necessary for every analysis. Original quantity and price values should be retained for interpreting business results, and scaling should be fitted only on the training data when building predictive models.

7. Python Preprocessing Pipeline:

A preprocessing pipeline combines data inspection, cleaning, and validation into an organised sequence of steps. Python and pandas are used to prepare the Online Retail Dataset for further analysis.

7.1 Complete Preprocessing Code:

```
import pandas as pd
import numpy as np

# 1. Load the dataset
df = pd.read_excel("Online Retail.xlsx")

# 2. Remove exact duplicate rows
df = df.drop_duplicates().copy()

# 3. Convert InvoiceDate to datetime
df["InvoiceDate"] = pd.to_datetime(
    df["InvoiceDate"], errors="coerce"
)

# 4. Inspect missing values
print("Missing values before cleaning:")
print(df.isnull().sum())

# 5. Identify cancelled invoices
cancelled = df["InvoiceNo"].astype(str).str.startswith("C")

# 6. Create a dataset for ordinary sales
sales_df = df[~cancelled].copy()

sales_df = sales_df[
    (sales_df["Quantity"] > 0) &
```

```
    (sales_df["UnitPrice"] > 0) &
    (sales_df["InvoiceDate"].notna())
].copy()

# 7. Calculate total sales value
sales_df["TotalSales"] = (
    sales_df["Quantity"] * sales_df["UnitPrice"]
)

# 8. Validate the cleaned dataset
print("Cleaned dataset shape:", sales_df.shape)
print("Remaining duplicates:", sales_df.duplicated().sum())
print("Remaining missing values:")
print(sales_df.isnull().sum())

# 9. Save the cleaned dataset
sales_df.to_csv("cleaned_online_retail.csv", index=False)
```

7.2 Explanation of the Pipeline:

The pipeline loads the dataset, removes exact duplicates, converts the date column, and inspects missing values. It then separates cancelled invoices and filters out records with non-positive quantities or prices and invalid dates.

A **TotalSales** column is calculated by multiplying quantity by unit price. Finally, the cleaned dataset is validated and saved as a CSV file for future analysis.

Missing customer IDs and product descriptions are reported rather than automatically removed. They should be handled according to the requirements of each analysis.
This pipeline provides a repeatable method for preparing transaction data. Outlier detection and scaling can be applied as additional steps when required for specific analytical tasks.

8. Impact on Logistics Analysis:

Data cleaning and preprocessing improve the reliability of data used for logistics-related analysis. Missing values, duplicate records, invalid

transactions, and extreme observations can affect calculations and lead to misleading conclusions.

The preprocessing steps in this project support the following activities:

- **Demand Analysis:** Clean transaction quantities help identify product demand patterns.
- **Sales Analysis:** Removing duplicate records and separating cancelled transactions improves the reliability of sales calculations.
- **Customer Analysis:** Identifying missing customer IDs helps determine which records are suitable for customer segmentation.
- **Forecasting:** Correct date formats and reliable historical records support time-based analysis and demand forecasting.
- **Machine Learning:** Normalization and standardization prepare numerical features for distance-based algorithms such as K-Means clustering.

However, the UCI Online Retail Dataset does not contain actual delivery times, vehicle details, route distances, or transportation costs. Therefore, it demonstrates data processing methods rather than directly measuring SwitRoute’s delivery or transportation performance.

For real-world implementation, SwiftRoute would need accurate operational data from its order management, warehouse, vehicle tracking, and delivery systems.